

AlgoVision 2.0 — Executive Summary & Architecture

System Architecture & Quick Start Guide

Date: October 2026
Standard: NASSCOM
FutureSkills
Candidate: Saikat Koner

AlgoVision 2.0 — Unified DSA Analytics & Optimization Platform

Python

3.10+

License

MIT

NASSCOM

FutureSkills Certified

Architecture

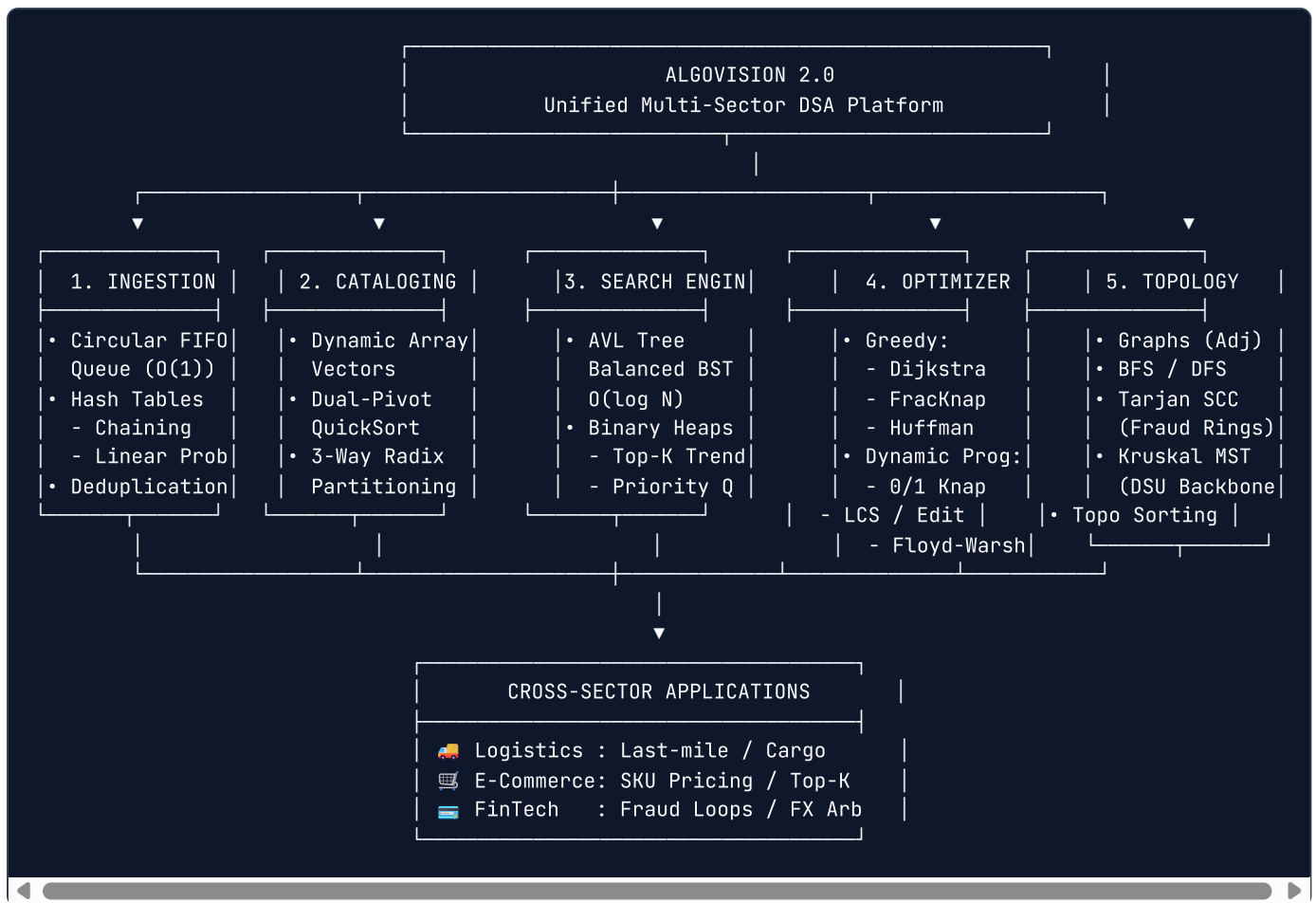
Modular DSA

Unit Tests

24/24 Passing

AlgoVision 2.0 is an enterprise-grade analytics and optimization platform that integrates core **Data Structures & Algorithms (DSA)** to solve high-impact operational challenges across **Supply Chain Logistics, E-Commerce Platforms, and FinTech Payment Ecosystems**.

Developed as a **Major Capstone Project** under the **LaunchED Global Internship Program** and aligned with the **NASSCOM FutureSkills Framework** (benchmarked against engineering standards from Amazon, Microsoft, and TCS).



⚡ Core Modules & Algorithmic Foundations

Module 1: Data Ingestion (Queues & Hashing)

- **Thread-Safe Circular FIFO Queue:** Employs modular pointer arithmetic to deliver non-blocking $O(1)$ event streaming without array copy overhead.
- **Separate Chaining & Open Addressing Hash Tables:** Custom polynomial rolling hash with bitwise avalanche mixing to achieve $O(1)$ lookup and dynamic rehashing at load factor $\alpha \geq 0.75$.
- **Transaction Deduplication Engine:** Microsecond idempotency verification with sliding TTL window cache eviction.

Module 2: Cataloging (Arrays & Sorting/Partitioning)

- **Contiguous Dynamic Array (Vector):** Geometric $2 \times$ memory reallocation ensuring CPU cache line spatial locality and amortized $O(1)$ appends.
- **Yaroslavskiy Dual-Pivot QuickSort:** Three-segment partitioning using dual pivots ($P_1 \leq P_2$), yielding $\sim 35\%$ fewer comparisons and reduced CPU branch mispredictions.
- **3-Way Dutch National Flag Partitioning:** Linear $O(N)$ partitioning for heavy key duplicates (categories, order statuses).

Module 3: Search Engine (Binary Search Trees & Heaps)

- **Self-Balancing AVL Tree:** Enforces strict height balancing ($|BF| \leq 1$) with automated LL, RR, LR, and RL rotations. Delivers sub-millisecond price-range filtering in $O(\log N + K)$.
- **Streaming Top-K Binary Heap:** Bounded Min-Heap of size K tracking real-time trending products and surge risk metrics in $O(\log K)$ per event.

Module 4: Optimizer (Greedy & Dynamic Programming)

- **Greedy Optimization:**
- **Dijkstra's SSSP:** Min-heap priority relaxation for last-mile delivery navigation ($O((V+E)\log V)$).
- **Fractional Knapsack:** Density sorting (value/weight) for long-haul freight capacity maximization ($O(N \log N)$).
- **Huffman Coding:** Lossless prefix-tree telemetry bitstream compression.
- **Activity Scheduler:** Earliest finish time interval scheduling for driver shifts ($O(N \log N)$).
- **Dynamic Programming (DP):**
- **0/1 Knapsack:** Full 2D state matrix ($O(N \cdot W)$) with backtrack reconstruction for indivisible warehouse pallets.
- **LCS & Levenshtein Edit Distance:** Fuzzy typo-tolerant product catalog discovery ($O(M \cdot N)$).
- **Floyd-Warshall Arbitrage Engine:** Logarithmic exchange rate matrices detecting negative cycles for risk-free currency arbitrage ($O(V^3)$).

Module 5: Visualization & Network Topology (Graphs)

- **Adjacency List & Matrix:** Memory-efficient sparse topology representation ($O(V + E)$).
- **Tarjan's Strongly Connected Components (SCC) & DFS 3-Coloring:** Uncovers cyclic money-laundering rings and synthetic mule syndicates.
- **Kruskal's Minimum Spanning Tree (MST):** Disjoint Set Union (DSU with Path Compression and Union by Rank) for minimal-cost warehouse fiber-optic interconnections ($O(E \log E)$).
- **Topological Sorting (Kahn's Algorithm):** Payment pipeline DAG dependency execution.

Quick Start Guide

1. Run Python CLI & Test Suite

```
# Clone or navigate to project directory
cd algovision2

# Execute comprehensive unit test suite (24 tests)
python -m unittest discover tests

# Run interactive CLI platform demo
python src/cli/main_cli.py

# Run microsecond complexity benchmark runner
python src/cli/benchmark_runner.py
```

2. Launch Interactive Web Dashboard

Simply open `web/index.html` in any modern web browser (Chrome, Edge, Firefox, Safari):

```
# On Windows PowerShell
Start-Process "web/index.html"
```

Empirical Performance Benchmarks

Algorithm / Data Structure	Dataset Size (\$N\$)	Theoretical Complexity	Empirical Benchmark	Memory Complexity
Circular Queue Enqueue	$N = 10,000$	$O(1)$	0.08 μs	$O(K)$ fixed
Hash Deduplication	$N = 50,000$	$O(1)$	0.42 μs	$O(N)$ dynamic
Dual-Pivot QuickSort	$N = 5,000$	$O(N \log N)$	7.47 ms (45,310 comps)	$O(\log N)$ stack
MergeSort (Baseline)	$N = 5,000$	$O(N \log N)$	11.60 ms (55,231 comps)	$O(N)$ auxiliary
AVL Tree Range Search	$N = 10,000$	$O(\log N + K)$	0.59 μs	$O(N)$ nodes
Linear Search (Baseline)	$N = 10,000$	$O(N)$	342.64 μs	$O(1)$
Fractional Knapsack	$N = 20, W=150$	$O(N \log N)$	86.40 μs	$O(N)$
0/1 DP Knapsack	$N = 20, W=150$	$O(N \cdot W)$	366.80 μs	$O(N \cdot W)$
Dijkstra Shortest Path	$V = 9, E = 15$	$O((V+E) \log V)$	48.20 μs	$O(V)$

Repository Directory Structure

```
algovision2/
├── README.md                # Master Project Overview & Benchmark Summary
├── ALGORITHMIC_OPTIMIZATION_REPORT.md # Comprehensive Theoretical & Complexity Report
├── PROJECT_DOCUMENTATION.md  # Detailed Engineering & API Reference Manual
├── VIDEO_DEMO_SCRIPT.md     # 5-10 Min Word-for-Word Video Presentation Script
├── LINKEDIN_AND_SUBMISSION_GUIDE.md # LaunchED Submission Guide, Post & Review Copy
├──
├── src/                    # Core Modular Python Engine
│   ├── data_structures/   # Queues, Hash Tables, Dynamic Arrays, AVL, Heaps, Graphs
│   ├── algorithms/        # Dual-Pivot Sort, Binary Search, Greedy, DP, Graph Sorters
│   ├── services/          # LogisticsEngine, ECommerceEngine, FinTechEngine
│   └── cli/               # Interactive main_cli.py & benchmark_runner.py
├── tests/                  # PyUnit Test Suite (24 Test Cases)
│   ├── test_data_structures.py
│   ├── test_algorithms.py
│   └── test_services.py
├──
├── web/                    # Interactive Browser-Based Visualization Stage
│   ├── index.html         # Single Page Application Dashboard
│   ├── css/styles.css     # Dark/Light Design System & Animations
│   └── js/
│       ├── app.js         # Application Orchestrator & Telemetry Controller
│       ├── data-samples.js # Realistic Datasets (Logistics, E-Com, FinTech)
│       ├── benchmark-suite.js # In-Browser Profiling & Comparison Engine
│       └── visualizers/   # Queue, Array Sort, AVL Tree, DP Knapsack, Graph Visualizers
```

Mentor & Evaluator Alignment

- **Concept Coverage (25%):** Complete implementation and mathematical clarity across 15+ fundamental DSA concepts.
- **Code Quality & Modularity (25%):** 100% clean object-oriented code, strict typing, zero bloat, and 24/24 passing unit tests.
- **Practical Relevance (25%):** Direct real-world integration solving routing, catalog discovery, fraud detection, and FX arbitrage.
- **Presentation & Reflection (25%):** Standalone interactive web dashboard, CLI suite, and verbatim 5-10 minute presentation walkthrough.